

Rapport de Travaux Pratiques – Séance 6

APIs REST HTTP, Fiabilité et Robustesse

Filière Cybersécurité – Cycle Ingénieur 1A
EUROMED Fes

14 juin 2026

Table des matières

1	Introduction et Objectifs	2
2	Architecture Globale et Fiabilité Mathématique	2
3	Implémentation Logicielle	2
3.1	Serveur HTTP Résilient (extract)	2
3.2	Client Défensif avec Traçabilité	2
4	Analyses Cybersécurité et Réponses aux Risques	2
5	Conclusion	3

1 Introduction et Objectifs

Ce travail pratique s'inscrit dans le cadre du module *Applications Réparties et Cybersécurité*. L'objectif est de concevoir une architecture de communication HTTP synchrone robuste à l'aide de la bibliothèque standard Python, tout en intégrant des mécanismes avancés de tolérance aux pannes transitoires et de sécurité applicative.

Les trois axes principaux de cette étude sont :

- La validation stricte des contrats de données entrantes (anti-injection et intégrité).
- L'implémentation de la fiabilité client (Timeouts, Reprises et Jitter).
- La mise en œuvre de l'observabilité système (Traçabilité via identifiants uniques).

2 Architecture Globale et Fiabilité Mathématique

Lorsqu'un client interroge un microservice, une panne réseau temporaire ou une surcharge du serveur peut entraîner un déni de service. Pour éviter la saturation du réseau lors des reconnections, nous implémentons un algorithme de **Backoff Exponentiel** avec **Jitter**.

La formule de calcul du délai d'attente avant la tentative n est définie mathématiquement par :

$$\text{délai_théorique} = \min(\text{base_delay} \times 2^n, \text{max_delay}) \quad (1)$$

Afin d'éviter que tous les clients déconnectés ne réessayent exactement au même instant (*Retry Storm*), on applique une variation aléatoire ou *Jitter* :

$$\text{délai_final} = \text{délai_théorique} \times \text{random}(0.5, 1.5) \quad (2)$$

3 Implémentation Logicielle

3.1 Serveur HTTP Résilient (extract)

Le serveur valide le jeton d'authentification et filtre rigoureusement le schéma JSON reçu. Tout champ superflu ou type incorrect engendre immédiatement un code d'erreur 400 Bad Request.

3.2 Client Défensif avec Traçabilité

Le client intègre un en-tête **X-Request-ID** généré par UUIDv4 permettant d'assurer la traçabilité de bout en bout de la requête dans les journaux d'audit du serveur.

4 Analyses Cybersécurité et Réponses aux Risques

Le tableau ci-dessous dresse la matrice des risques de cette architecture de communication :

Menace / Risque	Impact	Contre-mesure appliquée
Déni de Service (DoS)	Saturation serveur	Backoff Exponentiel et <i>Jitter</i>
Mass-Assignment	Injection de variables	Validation stricte du schéma JSON
Replay / Non-traçabilité	Usurpation d'action	Injection obligatoire du X-Request-ID

TABLE 1 – Matrice de Sécurité de l'API HTTP

5 Conclusion

Ce TP démontre qu'une couche de communication ne doit jamais faire confiance aveuglément au réseau ou aux données tierces. Les mécanismes mis en place garantissent qu'en cas de panne, le système se dégrade de manière gracieuse sans compromettre l'infrastructure.